Reactive Time-Domain Motion Planning via a Configuration-Independent Double-Integrator Backbone

Anonymous Authors

***Abstract*—Traditional motion-planning pipelines separate geometric path generation from time parameterization, making updates costly when obstacles or task objectives change. This paper presents a local predictive motion planner formulated directly in time. Planning on a virtual double-integrator backbone rather than a plant linearization keeps the state-transition matrix A_d constant across configurations. Prediction matrices are therefore precomputed offline, and online planning becomes a convex quadratic program (QP) with fixed dynamics structure.**

The planner jointly enforces velocity and acceleration limits and updates the trajectory without rebuilding a geometric path. This exchanges static-path time optimality for reactive constrained generation: time-optimal parameterization is 3–3.5× faster with position tracking, while an analytic velocity-law reference narrows the ratio to 1.06–1.32×. On a fixed grid, the planner generates $(q_d, \dot{q}_d)$ with hard-bounded, piecewise-constant $\ddot{q}_d$. A triple-integrator extension bounds jerk when continuous acceleration is required; in autonomous-vehicle lateral planning, it produces smooth steering within the tested actuator-rate envelope. FR3 benchmarks show fixed-size reactive updates and exact satisfaction of the declared kinematic bounds. On an Intel Xeon w7-2475X, the single-threaded Python harness records coupled-QP solve p95 of 3.87–4.49 ms and full-cycle p95 of 6.60–7.70 ms; full-cycle maxima of 12.26–18.48 ms identify the tail-latency target for hard 100 Hz deployment.

***Index Terms*—Motion planning, model predictive control, reactive replanning, real-time trajectory generation, double integrator, obstacle avoidance, time parameterization.**

I. Introduction

Modern robotic systems deployed in unstructured or human-shared environments must track planned trajectories with high precision while remaining responsive to dynamic workspace changes. Classical motion planning pipelines often address these demands sequentially. A geometric planner, such as OMPL [1], first searches for a collision-free path in configuration space. A separate time-parameterization stage, such as Time-Optimal Trajectory Generation (TOTG) [2], then maps this spatial path into a timed trajectory through an arc-length-to-time transformation $s \to t$. This separation is effective for static scenes, but it creates a structural latency issue in reactive settings: when obstacles or task objectives change, the system must update the geometric path, the time law, or both before a new command can be executed. Even when incremental replanners reduce this latency, the planning stack remains organized around a batch path-then-time interface rather than a fixed-rate trajectory update.

Manuscript submitted to IEEE ICRA 2027. Author information omitted for double-anonymous review.

This paper asks whether the timed trajectory can instead be generated directly, at the planning rate, while preserving convex kinematic constraints. The answer rests on an architectural choice rather than a property of the double integrator itself: an isolated double integrator is of course linear time-invariant, but a model predictive planner built on the *physical* plant must re-linearize and rebuild its prediction matrices at every operating point. By planning instead on a virtual double-integrator backbone, the state-transition matrix A_d is held constant across all robot configurations, so the horizon prediction matrices can be computed once offline and reused at every replanning cycle. The online problem is reduced to a convex Quadratic Program (QP) with fixed dynamics structure; only the current state, reference terms, and active constraint rows change. The contribution is a planner that, from this offline-precomputed and configuration-independent prediction structure, obtains hard kinematic feasibility and non-halting reactive replanning *together*, at the cost of one convex QP per cycle—a combination a path-then-time pipeline provides only in separate stages. Concretely, this substitutes the geometric $s \to t$ interface by a fixed-structure time-domain QP; it is not the observation that a double integrator is linear.

The idea is inspired by a two-layer predictive impedance-control architecture for pHRI [3], where dynamic cancellation exposes a constant double-integrator residual. That architecture has since been verified on a physical UR5e (held-load error 1.5 mm $\to$ 0.023 mm with the estimator) [3]. Here the double integrator is a *planning backbone*: the planner optimizes reference trajectories for a downstream controller (computed-torque, impedance, admittance, or other), so the contribution is a configuration-independent time-domain parameterization enabling fast repeated QP updates.

The resulting framework is a **local predictive motion planner**; we refer to it throughout as the **DI-QP planner** (the double-integrator backbone solved as a receding-horizon QP), and use "time-first" only when contrasting its time-domain structure with the "path-first" TOTG pipeline. It performs no global search; obstacle avoidance is handled locally, by linearized safe-corridor rows or artificial-potential-field reference deflection. The framework's prediction structure is independent of robot configuration (growing only with horizon, dimension, and active rows); it directly optimizes time-indexed positions, velocities, and accelerations on a fixed grid via one QP in place of the separate spatial-to-temporal stage, producing continuous C^1 output extended to piecewise-

jerk when needed.

II. Framework and System Structure

Let $q \in \mathbb{R}^n$ denote robot configuration and $p = f(q)$ task position. The planner generates $(q(t), \dot{q}(t), \ddot{q}(t))$ satisfying waypoint, joint, velocity, acceleration, and obstacle constraints; actuator commands are left to a downstream tracking controller. The trajectory space is parameterized by a virtual double-integrator backbone,

$$\ddot{q} = u, \qquad x = \begin{bmatrix} q^\top & \dot{q}^\top \end{bmatrix}^\top, \tag{1}$$

whose state contains planned positions and velocities. It is a configuration-independent trajectory generator that makes acceleration a decision variable and therefore keeps velocity and acceleration bounds linear.

A. Double-Integrator Backbone from Dynamic Cancellation

The same backbone can also be derived from the standard manipulator dynamics $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau$. With the computed-torque (inverse-dynamics) feedforward law—the same cancellation used in the pHRI architecture of [3]—

$$\tau = M(q)u + C(q,\dot{q})\dot{q} + g(q), \tag{2}$$

substitution gives the backbone in (1), and hence

$$\dot{x} = \begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ I \end{bmatrix} u. \tag{3}$$

The present work uses this residual model as a planning backbone rather than as an impedance-regulation controller.

B. Exact ZOH Discretization and the Constant A_d Property

Because the continuous system matrix is nilpotent, $A_c^2 = 0$, exact ZOH discretization over Δt gives

$$A_d = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \qquad B_d = \begin{bmatrix} \frac{\Delta t^2}{2} \\ \Delta t \end{bmatrix}. \tag{4}$$

Unlike a physical plant linearization, this A_d is independent of configuration, operating point, and joint index. Therefore the horizon prediction matrices

$$\Phi = \begin{bmatrix} A_d \\ A_d^2 \\ \vdots \\ A_d^N \end{bmatrix}, \qquad \Gamma = \begin{bmatrix} B_d & 0 & \cdots & 0 \\ A_d B_d & B_d & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ A_d^{N-1} B_d & \cdots & A_d B_d & B_d \end{bmatrix} \tag{5}$$

are precomputed once and reused at every planning cycle. Online rollout then reduces to

$$X = \Phi x(k) + \Gamma U, \tag{6}$$

decoupling prediction cost from robot configuration.

III. Receding-Horizon Optimization

A. Optimization Problem Structure

At each planning time step, the planner solves for the optimal acceleration profile sequence across the prediction horizon:

$$U = \begin{bmatrix} u^\top(0) & u^\top(1) & \cdots & u^\top(N-1) \end{bmatrix}^\top \in \mathbb{R}^{nN}. \tag{7}$$

where n is the number of joints and N is the horizon length (e.g., $nN = 7 \times 20 = 140$ decision variables for the 7-DOF FR3 with $N = 20$). The predicted trajectory follows (6), which is affine in U; therefore kinematic bounds and linearized obstacle rows remain convex. Absent obstacle coupling, the Hessian is block-diagonal and the problem separates into n independent per-joint QPs of size N.

B. Cost Function: Tracking and Smoothness

The objective function minimizes the weighted sum of tracking deviations from a goal state $x_{\text{goal}} = [q_{\text{goal}}, 0]^\top$ and control effort. We write the stage cost with the conventional $\frac{1}{2}$ scaling,

$$J(U) = \frac{1}{2} \left(\Phi x(k) + \Gamma U - \mathbf{1}_N \otimes x_{\text{goal}}\right)^\top \bar{Q} \times \left(\Phi x(k) + \Gamma U - \mathbf{1}_N \otimes x_{\text{goal}}\right) + \frac{1}{2} U^\top \bar{R} U. \tag{8}$$

so that collecting the terms depending on U gives

$$\min_U \ \frac{1}{2} U^\top H U + h^\top U \tag{9}$$

where:

$$H = \Gamma^\top \bar{Q} \Gamma + \bar{R}, \qquad h = \Gamma^\top \bar{Q}\, x_{\text{free,err}} \tag{10}$$

and $x_{\text{free,err}} = \Phi x(k) - \mathbf{1}_N \otimes x_{\text{goal}}$ is the unforced deviation from the goal; h is the gradient at $U = 0$, i.e. the linear-cost vector passed to OSQP. The remaining definitions are: the full goal state $x_{\text{goal}} = [q_{\text{goal}}, 0]^\top$ replicated across all N horizon steps; stage weights $Q = \text{blkdiag}(K_q, D_q)$ on position/velocity error; control cost R regularizing acceleration; terminal weight $Q_f = \gamma Q$ ($\gamma \approx 5$–10); and horizon-stacked $\bar{Q} = \text{blkdiag}(Q, \ldots, Q, Q_f)$, $\bar{R} = I_N \otimes R$.

Because H is the sum of a positive semi-definite matrix (from $\bar{Q}$) and a positive definite matrix (from $\bar{R}$ with $R \succ 0$), the QP is **strictly convex** for all configurations and all time steps. A unique global minimizer exists and is found efficiently by operator-splitting solvers such as OSQP [4] with warm-starting, converging in under 0.5 ms for the decoupled box-constrained QP and in a few milliseconds for the coupled-obstacle QP (Section VI-B).

Weight tuning guidance. K_q sets convergence aggressiveness, $D_q \approx 2\sqrt{K_q}$ gives critical damping when velocity-error weighting is used, R trades smoothness against speed, and $\gamma = 5$–10 compensates the finite horizon at $N \geq 20$. The static FR3 B1–B4 benchmark uses $K_q = 50$, $D_q = 0$, $R = 0.05\, I$, $\gamma = 8$; the FR3 dynamic-obstacle study uses $K_q = 6$, $D_q = 8$, $R = 0.05\, I$, $\gamma = 8$; and the jerk-bounded lateral planner uses $w_q = 20$, $w_a = 0.2$, $w_r =$

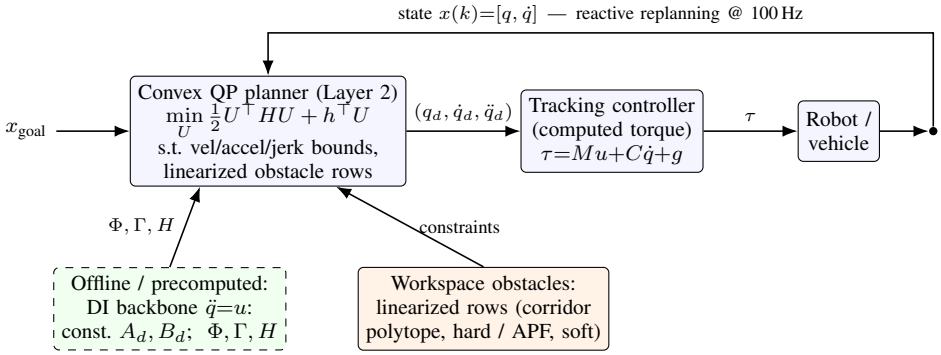


Fig. 1. Reactive predictive motion-planning architecture. The planner runs on a virtual *double-integrator backbone* ($\ddot{q} = u$), whose nilpotent dynamics give a configuration-independent constant A_d, so the horizon prediction matrices Φ, Γ and the QP Hessian H are precomputed *offline* (left, dashed). Online, a convex QP (Layer 2) optimizes the acceleration profile U subject to velocity/acceleration/jerk bounds and linearized workspace-obstacle rows (convex corridor polytope, hard; or APF goal deflection, soft), emitting a time-indexed trajectory $(q_d, \dot{q}_d, \ddot{q}_d)$ to a downstream tracking controller and plant. Current state $x(k)$ and updated obstacles re-enter every cycle, giving fixed-size reactive updates without the stop-and-rebuild of path-first pipelines; Section VI reports the measured 100 Hz timing distribution and tail.

0.02, $\gamma = 10$. These settings match `benchmark_fr3.py`, `fr3_dynamic_obstacle.py`, and `di_lateral.py` in the frozen benchmark snapshot.

C. Kinematic Hard Constraints

Physical joint limits are enforced as hard inequality constraints on the predicted state and input sequences. For each joint i and each step k along the horizon:

$$\begin{aligned} -\dot{q}_{i,\max} \le \dot{q}_i(k) \le \dot{q}_{i,\max}, \qquad & k = 1, \dots, N, \\ -u_{i,\max} \le u_i(k) \le u_{i,\max}, \qquad & k = 0, \dots, N-1. \end{aligned} \tag{11}$$

Since the predicted states are affine in U by (6), the bounds in (11) translate to linear inequalities $C_{\text{kin}}U \le d_{\text{kin}}$ assembled once from the precomputed Γ matrix and fixed online. Optionally, joint position limits $q_{i,\min} \le q_i(k) \le q_{i,\max}$ can be appended with no additional computational overhead.

Because the velocity bounds are imposed without slack, feasibility of the QP assumes the current state lies inside the velocity polytope; if a disturbance drives $\dot{q}(k)$ outside it, the box rows should be softened on the first horizon step or a terminal maximal-braking-to-rest set appended to recover recursive feasibility (see Section VIII).

IV. Incorporating Workspace Obstacle Avoidance

The linearity of the predicted joint positions with respect to U makes it straightforward to incorporate obstacle avoidance constraints without breaking convexity. Two complementary methods are presented.

A. Convex Corridor Polytope Constraints (Hard Constraints)

Cartesian obstacles detected by perception systems are represented as half-space inequalities in task space:

$$A_{\text{obs}}\, p(k) \le b_{\text{obs}} \tag{12}$$

where $p(k) \in \mathbb{R}^3$ is the predicted end-effector (or link) position at step k. Using the robot's forward kinematics linearized at the current operating point via the translational Jacobian $J_v(q_0)$:

$$p(k) \approx p_0 + J_v(q_0)\big(q(k) - q_0\big) \tag{13}$$

the workspace constraint in (12) becomes:

$$A_{\text{obs}} J_v(q_0)\, \Delta q(k) \le b_{\text{obs}} - A_{\text{obs}} p_0 \tag{14}$$

where $\Delta q(k) = q(k) - q_0$ is linear in U through Γ. To make the QP row explicit, let $E_{q,k} \in \mathbb{R}^{n \times 2nN}$ be the selector that extracts the n-dimensional joint-position block of the stacked state X at horizon step k (a row of $n \times n$ identity blocks, zero elsewhere), so

$$q(k) = E_{q,k}\big(\Phi x(k_0) + \Gamma U\big). \tag{15}$$

Define $C_{\text{obs},k} = A_{\text{obs}} J_v(q_0) E_{q,k} \Gamma$. Then each obstacle row has the affine form

$$\begin{aligned} C_{\text{obs},k} U \le\ & b_{\text{obs}} - A_{\text{obs}} p_0 \\ & - A_{\text{obs}} J_v(q_0)\big(E_{q,k}\Phi x(k_0) - q_0\big) =: d_{\text{obs},k}. \end{aligned} \tag{16}$$

Thus the free-response term is absorbed into the right-hand side, and the obstacle constraint becomes a set of affine inequalities in U, fully compatible with the convex QP structure.

Approximation quality. Because $J_v(q_0)$ is held constant over the horizon, these rows are a first-order local approximation—exact near q_0, increasingly inaccurate away from it. Re-linearizing every cycle refreshes it; a conservative half-space additionally tightens each row by a bound on the linearization error. From the second-order Taylor remainder, with $H_p(q_0)$ the position Hessian,

$$\|p(q) - (p_0 + J_v(q_0)\Delta q)\| \le \tfrac{1}{2}\|H_p(q_0)\|_F \|\Delta q\|^2, \tag{17}$$

so an obstacle row $a^\top p \le b$ tightens to $a^\top(p_0 + J_v(q_0)\Delta q(k)) \le b - \|a\|\,\varepsilon_p$ with ε_p set from (17). For the FR3, $\|H_p\|_F$ sampled at 5000 configurations has mean 1.64, 95th-percentile 1.96 m/rad^2, and the measured end-horizon error grows quadratically with joint-speed×horizon (7.1 mm at 25% speed over 0.1 s, 106.4 mm at 50% over 0.2 s, 1.03 m

at full speed over 0.4 s)—well below the Frobenius bound's worst case (≈1.0 m at 50% speed), since the executed motion does not excite the worst-case curvature. In practice the margin should be set from the *measured* drift rather than the conservative Frobenius bound, with the horizon shortened for fast motions; at modest speeds a centimetre-scale margin suffices.

Accuracy of the constraint guarantee. The QP guarantees satisfaction of the *linearized* safe-corridor constraints: no predicted trajectory penetrates the declared linearized obstacle polytope. A true nonlinear collision-clearance guarantee additionally requires conservative tightening by ε_p, nonlinear collision checking of the accepted rollout, or both; for non-convex environments, standard safe-corridor decomposition [5] can be applied upstream.

Multiple obstacles. Method A appends M independent half-space sets, keeping convexity at mildly increasing solver cost; if rows conflict the QP becomes infeasible, resolved by per-obstacle slacks $s_m \geq 0$ with penalty $\mu\|s\|^2$. Method B sums individual APF repulsion fields, avoiding infeasibility but inheriting the local-minimum risk of scalar APF methods.

B. Artificial Potential Field Target Deflection (Soft Constraint)

For scenarios requiring maximum solver throughput, an alternative soft-constraint approach using Artificial Potential Fields (APF) [6] is available. An obstacle at position p_{obs} with influence radius ρ_0 and repulsive gain η generates a task-space repulsive potential:

$$\mathcal{U}_{\text{rep}}(p) = \begin{cases} \frac{1}{2}\eta\left(\frac{1}{\|p-p_{\text{obs}}\|} - \frac{1}{\rho_0}\right)^2, & \|p-p_{\text{obs}}\| \leq \rho_0, \\ 0, & \text{otherwise.} \end{cases} \tag{18}$$

The negative gradient $g_{\text{rep}}(p) = -\nabla\mathcal{U}_{\text{rep}}(p)$, treated as a synthetic direction field rather than a physical force, is normalized as $\hat{g}_{\text{rep}} = g_{\text{rep}}/(\|g_{\text{rep}}\|+\epsilon)$ and mapped to a heuristic joint-space target shift via the Jacobian pseudo-inverse,

$$\delta q_{\text{obs}} = K\, J_v^+(q)\, \hat{g}_{\text{rep}}, \tag{19}$$

where the scalar gain $K > 0$ absorbs all dimensional scaling. (In the simulations below the same idea is implemented as a velocity/reference deflection with an optional tangential component to avoid stalls; in both cases APF changes the reference rather than adding a hard row.) The target fed to the cost function is then updated online:

$$q_{\text{target}}(k) = q_{\text{goal}} + \delta q_{\text{obs}}. \tag{20}$$

This bypasses inequality rows entirely, preserving the minimal QP structure and maximizing solve speed. The trade-off is that obstacle avoidance is enforced softly through the cost function rather than guaranteed through hard constraints; APF-based avoidance is also susceptible to local minima in complex environments (see Section VIII).

Local minima mitigation. When the APF gradient vanishes the planner stalls; three escape strategies are available—waypoint injection after a no-progress timeout, a small random perturbation to q_{target}, or a hybrid switch to the Method-A hard polytope (with slack) on stall—of which waypoint injection is preferred since it adds no solver overhead. The two methods are complementary: hard polytope constraints for static infrastructure, APF deflection for dynamic obstacles.

V. Architectural Properties

A. Constant-Structure MPC, Fixed-Size Updates, and C^1 Output

All dynamics prediction matrices (Φ, Γ, H) are precomputed offline and never rebuilt online; obstacle rows are re-assembled each cycle but still project J_v through the fixed Γ. Each cycle thus executes a QP whose dynamics structure is fixed at initialization. Per-cycle work consists of one QP update and solve for a given active set—growing with the obstacle-row count but independent of configuration and free of prediction-matrix rebuilds. This is distinct from configuration-dependent MPC (which recomputes prediction matrices at each linearization point) and from batch planning (TOTG, TOPP-RA, where any workspace change triggers a full rebuild). The constant-A_d property makes the update size predictable; it does not itself establish a hard execution-time bound, so Section VI reports p95 and maximum latency separately. Because the framework has no geometric path—positions, velocities, and accelerations along the horizon are generated simultaneously in one QP—the $s \leftrightarrow t$ conversion stage and its singularities (Section VI-A) are eliminated by construction. And because joint acceleration u_i is the direct optimization variable under hard bounds, the output $(q_d, \dot{q}_d)$ is C^1 with $\ddot{q}_d$ hard-bounded and piecewise-constant (held over each zero-order-hold control cycle); full C^2 continuity requires jerk as the optimization variable, as in the extension below.

B. Piecewise-Jerk Triple-Integrator Extension

When continuous acceleration is required—to reduce torque-rate excitation, satisfy comfort limits, or bound steering-rate in vehicle applications—the same constant-A_d principle extends to a triple-integrator backbone that optimizes jerk $\dddot{q}_i = j_i$ on the state $z_i = [q_i, \dot{q}_i, \ddot{q}_i]^\top$. The continuous system matrix is again nilpotent ($A_c^3 = 0$), so exact ZOH discretization gives the configuration-independent

$$A_d^{(3)} = \begin{bmatrix} 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix}, \qquad B_d^{(3)} = \begin{bmatrix} \frac{1}{6}\Delta t^3 \\ \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix}. \tag{21}$$

so the prediction matrices remain precomputable offline. With jerk held constant per interval, $\ddot{q}(t+\tau) = \ddot{q}(t) + \tau j(t)$ keeps acceleration continuous across samples while jerk is piecewise-constant, raising output smoothness from C^1 to C^2 ($q_d \in C^2$, $\dot{q}_d \in C^1$, $\ddot{q}_d \in C^0$) with hard linear bounds now available on velocity, acceleration, and jerk.

The trade-off is a larger state and extra predicted-state rows, though the decision sequence is still length nN, now a jerk sequence $J = [j(0), \ldots, j(N-1)]$ rather than acceleration. The AV steering-rate benchmark (§VI.C) uses exactly this for the lateral channel, since steering rate depends on lateral jerk.

The rest of the paper uses the double-integrator backbone as the main manipulator planner because it is the smallest model that enforces velocity and acceleration limits directly and gives the fastest QP. The triple-integrator form is a drop-in extension when continuous acceleration or explicit jerk bounds are application-critical.

The unconstrained infinite-horizon limit of the planner is the standard LQR law $u = u_{ff} - K(x - x_r)$, situating the constrained, finite-horizon planner as its receding-horizon QP counterpart—mathematically close to linear MPC [9], but on a virtual trajectory state rather than a plant linearization; the robot model enters only through kinematics and workspace constraints [8], and the output is the reference trajectory $\mathcal{T} = \{q_d, \dot{q}_d, \ddot{q}_d\}$ for a downstream controller.

VI. Experimental Evaluation

We evaluate the proposed planner against a path-first Time-Optimal Trajectory Generation pipeline on a 7-DOF Franka FR3 manipulator and on an autonomous-vehicle steering task. Both planners receive identical waypoints and kinematic limits and emit $(q_d, \dot{q}_d, \ddot{q}_d)$ sampled at the same Δt. The accompanying `benchmarks_standalone.zip` inlines the shared planner modules and reproduces all non-MuJoCo tables with recorded source provenance and dependencies. The computed-torque execution table additionally requires the third-party Franka model/assets through `FR3_XML`. Wall-clock values are machine dependent; deterministic constraint and geometry metrics are reported separately. The TOTG reference uses the numerical-integration TOPP core—circular-blend path (Kunz–Stilman geometry [10]), time-optimal forward/backward sweep on $\dot{s}^2$ [7], then $s \to t$ inversion and resampling.

Scope of comparison. We compare against the path-first family (TOTG, TOPP-RA), a resolved-rate-plus-APF baseline, and a sampling-based MPPI baseline on the planar dynamic-obstacle test (Section VI-D). Online nonlinear MPC and CHOMP/TrajOpt are deferred to future work; the present planner's appeal is the constant-A_d precomputation and fixed-size online update, while richer nonlinear formulations address a broader planning model.

This section reports four evaluation axes: reactive latency, jerk/ smoothness, constraint fidelity, and the execution-time gap. On static paths, time-optimal path parameterization is faster; the fixed time-domain QP trades a small amount of that global time optimality for fixed-size reactive updates, explicit limit satisfaction, and smoother command profiles.

A. Path-First $t \leftrightarrow s$ Conversion Artifacts

The $s \to t$ reconstruction differentiates the geometric path,

$$\ddot{q}(t) = q'(s)\,\ddot{s} + \underbrace{q''(s)}_{\text{curvature vector}}\,\dot{s}^2.$$

Two pathologies follow from the standard circular-blend path representation. First, $q''(s)$ jumps at every blend seam (0 on a straight segment, $1/r$ on a circular arc), so the reconstructed $\ddot{q}$ is discontinuous, with the jump scaling as $\dot{s}^2$. Second, $t(s) = \int ds/\dot{s}$ is singular wherever $\dot{s} \to 0$ (rest points, tight near-reversal blends), making the uniform-time resample ill-conditioned. These artifacts are a property of the circular-blend path representation, not of time parameterization per se. Replacing circular blends with a C^2 path (e.g., cubic splines) removes them, as confirmed below. The time-first planner never forms s; its $\ddot{q}$ is the bounded decision variable, so neither pathology can arise.

In the $(s, \dot{s})$ phase plane (TOPP-RA style), the maximum-velocity curve and optimal profile $\dot{s}^*(s)$ dip toward zero at the tight blend—severely on the near-reversal, where $\dot{s} \to 0$ drives $t(s)$ singular.

B. FR3 Manipulator Results

Stresses are concentrated in two joints for interpretability, though all seven are planned. The decisive metric is the peak acceleration ratio $\|\ddot{q}\|_\infty / \ddot{q}_{\max}$: above 1 means the emitted trajectory violates the declared limit. Because the DI planner enforces this as a hard QP constraint, it is pinned at 1.000 with zero violations *by construction*; the informative comparison is how far TOTG departs from that bound.

Separating the decoupled and coupled cases, the single-threaded Python harness (OSQP tolerance 10^{-6}, warm-started, fixed KKT sparsity on the coupled update path) solves the 140-variable decoupled box-constrained QP (7 DOF, N=20) in 0.03–0.08 ms mean on the current workstation. Across three repeated fixed-sparsity runs, the 141-variable coupled obstacle QP has solve mean/p95/max ranges of 1.87–2.07/3.87–4.49/8.34–9.19 ms; including Python assembly and updates, the full-cycle ranges are 4.74–5.07/6.60–7.70/12.26–18.48 ms.

A horizon sweep $N = \{10, 20, 30, 40\}$ on the same coupled FR3 dynamic-obstacle problem (`sweep_N.py`) confirms why $N = 20$ (0.2 s) is the default. Because the scenario uses a velocity-law/APF reference, completion (1.83–1.86 s) and min EE clearance (≈0.30 m) are nearly insensitive to N; the dominant trend is computational, with coupled-QP solve p95 rising steeply from 1.0 ms (N=10) and 4.4 ms (N=20) to 20.4 ms (N=30) and 55.6 ms (N=40). For pure position-tracking waypoints $N = 10$ instead inflates time-to-goal, so $N = 20$ balances both regimes.

Comparison scope. The acceleration violation metric compares the trajectory as emitted by each planner—i.e., the command sequence as-sent to the controller, with no downstream smoothing or re-interpolation applied to either planner's output. In practice, some systems apply a final spline pass to the TOTG output before execution, which would reduce or eliminate the resampling-induced violations on B2 and B4; under such post-processing the DI planner's zero-violation advantage on those scenarios shrinks. The advantage on B3 (near-reversal singularity) and in the reactive scenarios (Sections VI.D–E) is independent of downstream smoothing.

Figure 2 plots the acceleration ratio over time for each scenario: the DI curve never crosses 1, while the TOTG curve spikes above it at blend seams.

In **B3 (near-reversal)** the blend radius collapses, $\dot{s} \to 0$, and the parameterization becomes singular: $\max(1/\dot{s}) \to \infty$

TABLE I
FR3 MANIPULATOR BENCHMARK RESULTS

Scenario	DI T [s]	TOTG T [s]	DI accel ratio	TOTG accel ratio	TOTG accel viol.	DI jerk RMS	TOTG jerk RMS
B1 point-to-point	2.02	**0.59**	1.000	1.000	0	**22**	143
B2 acute corner (tight blend)	1.92	**1.23**	1.000	**1.047**	2	36	133
B3 near-reversal	1.99	1.07	1.000	up to **3.13**	0–1	29	166
B4 dense (24 waypoints)	13.24	**3.83**	1.000	**1.445**	49	**54**	333

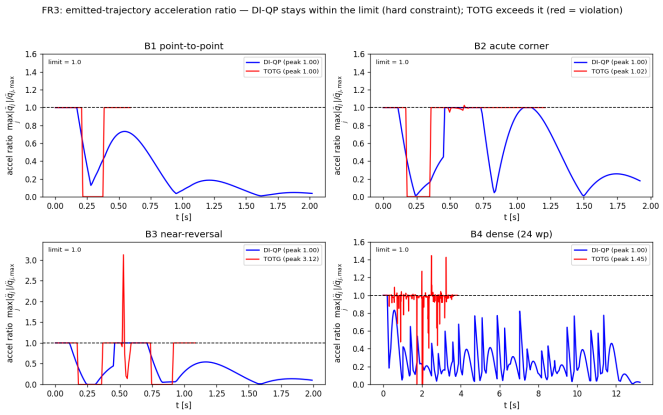


Fig. 2. FR3 emitted-trajectory acceleration ratio $\max_j |\ddot{q}_j|/\ddot{q}_{j,\max}$ over time, DI-QP (blue) vs TOTG (red), B1–B4; dashed line is the limit. DI-QP is pinned within the limit by hard constraints; TOTG exceeds it at blend seams (B2 $1.047\times$, B3 up to $3.13\times$, B4 $1.445\times$). The B3 spike is blend-dependent (drops to $1.00\times$ under TOPP-RA/C^2); the intrinsic path-first issues are the $\dot{s} \to 0$ singularity and curvature-discontinuous reconstruction, not the spike magnitude.

and the $s \to t$ conversion fails (flagged and capped by the harness). In **B4** the path-first command breaks the acceleration limit by 45% at 49 samples. The DI planner remains feasible and smooth throughout.

Robustness to the parameterizer (TOPP-RA). Re-running with reachability-based TOPP-RA [7] confirms the findings persist (acute corner $1.020\times$, dense path $1.12\times$, near-reversal still singular) while softening magnitudes (the B3 spike drops from $3.13\times$ to $1.000\times$), separating implementation-dependent spike size from the intrinsic path-first issues (curvature-discontinuous reconstruction, the $\dot{s} \to 0$ singularity). Replacing circular blends with a C^2 spline removes the B2/B4 violations entirely, so TOPP-RA on a C^2 path is a competitive baseline the DI planner does not strictly dominate on those metrics.

Time-optimality. TOTG is the lower bound by construction. With the smooth quadratic-cost setting, the DI completion-time ratios are $3.4\times$ in B1 and $3.5\times$ in B4. Supplying an analytic time-optimal velocity-law reference (velocity-dominant weights) recovers the gap to 1.06–$1.32\times$ while the DI-QP still preserves zero limit violations, at the cost of raising jerk RMS by roughly an order of magnitude—the velocity-law reference is thus an explicit speed-versus-smoothness selector. The DI planner remains feasible by construction with respect to the declared time-domain bounds, while path-first feasibility can be lost after circular-blend reconstruction and uniform-time resampling unless a smoother path representation or downstream post-processing is used.

C. Vehicle Steering-Rate Smoothness via the Lateral Backbone

The same backbone extends to autonomous-driving stacks that separate a Cartesian double integrator $\ddot{p} = u$ from steering/throttle control, recovering heading and curvature as $\theta = \operatorname{atan2}(\dot{y}, \dot{x})$, $\kappa = (\dot{x}\ddot{y} - \dot{y}\ddot{x})/(\dot{x}^2 + \dot{y}^2)^{3/2}$. The benchmark evaluates one concrete claim: applying the constant-A_d principle to the lateral channel—optimizing lateral jerk, the derivative that sets steering rate—produces a steering profile inside a representative actuator-rate envelope where the path-first command does not. On a lane-change maneuver at a fixed 13 m/s cruise speed with a 0.5 rad/s steering-rate cap, the jerk-bounded time-first extension stays within the cap (peak rate 0.130 rad/s) while the path-first command exceeds it on the double lane change (0.594 rad/s) because circular-blend curvature steps at path seams; since steering rate scales with speed at fixed path curvature, this margin is specific to that operating point. The result is a lateral-channel claim at a fixed cruise speed, complete and tested for that channel; vehicle feasibility beyond it comes from speed, acceleration, curvature, and steering-rate constraints on top of the parameterization.

D. Fixed-Size Reactive Replanning

The reactivity advantage is structural rather than implementation-dependent. A planar point robot must reach a goal while a disk obstacle crosses its straight-line path mid-execution. The DI planner recomputes an APF velocity reference from the obstacle's current position each 100 Hz cycle and re-solves the box-constrained QP, deflecting continuously. The path-first planner must halt, rebuild a detour path, and re-run TOPP from rest.

Both reach the goal collision-free, but with different cost structures. Across three runs on the test workstation, the DI planner's mean cycle latency is 0.118–0.130 ms and worst reaction latency is 1.92–2.22 ms: one warm-started QP update rather than a geometric path rebuild. Its completion time stays flat at 6.53 s regardless of added path-search latency. TOTG, which must halt and rebuild, degrades as that latency grows: completion rises from 6.28 s (halted 0.08 s) at zero added latency to 7.18 s (halted 1.26 s) at 400 ms. The reactive edge is decisive when replanning is expensive (full geometric search)

TABLE II
REACTIVE BASELINES ON THE PLANAR DYNAMIC-OBSTACLE TEST

Metric	DI-QP	MPPI
Reached goal	yes	yes
Completion time [s]	6.53	6.12
Min clearance [m]	0.485	0.483
Velocity limit enforced by	QP constraint	clamping
Cycles needing velocity clamp	**0**	166
Rollouts per cycle	1 QP	512×20
Mean cycle latency [ms]	**0.118–0.130**	1.886–1.979
Worst cycle latency [ms]	**1.92–2.22**	8.31–9.97

or obstacles are fast, and marginal when the rebuild is cheap: with zero replan latency, TOTG and DI have comparable completion times.

Comparison with a sampling-based reactive baseline. Since TOTG isolates the reactivity gap against path-first pipelines specifically, we also compare against a reactive method that, like ours, never halts: Model-Predictive Path Integral (MPPI) [14] on the same planar double integrator and moving obstacle (K=512 rollouts, H=20, λ=60, σ=4—the tightest-clearance setting in a temperature sweep; lower λ turns winner-take-all and fails to reach the goal). Both reach the goal with essentially identical safety, but the cost structure differs sharply (Table II): MPPI enforces the velocity limit only by clamping the applied command—on 166 cycles—whereas the DI-QP enforces it as a convex constraint and never clamps, at roughly $10\times$ lower mean compute; continuous reactivity is thus shared with MPPI, and pairing it with hard limit feasibility at QP-level cost is what the DI-QP adds.

A planar test also exposes the APF method's known failure mode: with a purely radial field, attractive and repulsive terms balance in front of an obstacle and the planner stalls (6.07 m short of the goal). A simple waypoint-injection rule—detect lack of progress, insert a lateral detour—recovers the same box-constrained QP to the goal (0.12 m residual) at the same clearance and limits, at negligible extra solve cost; this motivates using hard polytope constraints or explicit waypoint logic whenever progress stalls or clearance must be certified, with APF deflection reserved for fast reactive steering.

E. Scaling to the 7-DOF Arm with a Dynamic Obstacle

We exercise the full Section IV machinery on the 7-DOF FR3, where obstacle constraints couple the joints through the translational Jacobian: the arm reaches a joint-space goal while a spherical obstacle crosses the end-effector path, compared against a resolved-rate-plus-APF reactive controller without hard limits. Both avoid the obstacle and reach the goal, but the reactive baseline exceeds the joint velocity limit at 42 samples and acceleration at 346, whereas the QP holds every joint limit exactly and keeps the linearized half-space feasible (zero slack), at a min EE clearance of 0.296 m versus 0.339 m for the unconstrained baseline. Across three fixed-sparsity runs, coupled-QP solve mean/p95/max is 1.87–2.07/3.87–4.49/8.34–9.19 ms; including Python assembly and updates, the full cycle is 4.74–5.07/6.60–7.70/12.26–18.48 ms. The harness meets 100 Hz at p95, while the full-cycle maximum identifies the tail-latency target for a compiled implementation with reusable active-set or factorization state.

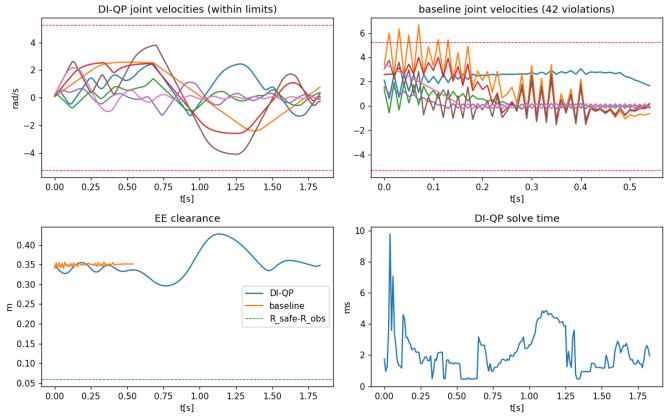


Fig. 3. 7-DOF FR3 dynamic obstacle. Top-left: DI-QP joint velocities, all within the limit. Top-right: resolved-rate+APF baseline, which exceeds the velocity limit. Bottom-left: end-effector clearance. Bottom-right: DI-QP per-cycle solve time.

TABLE III
RANDOMIZED FR3 DYNAMIC-OBSTACLE ROBUSTNESS

Metric	DI-QP	Reactive baseline
Success rate	100%	100%
Min EE clearance, mean ± std [m]	0.293 ± 0.018	0.328 ± 0.021
Worst min EE clearance [m]	0.261	0.266
Total joint velocity violations	**0**	618
Total joint acceleration violations	**0**	6889
Max slack used [m]	0.0000	—
QP solve p95 / max [ms]	6.15–6.66 / 25.96–33.10	—

F. Randomized FR3 Dynamic-Obstacle Robustness

To check that the dynamic-obstacle result is not tied to a single hand-picked configuration, we perturb the start and goal joint states, obstacle lateral offset, initial height, and crossing speed over 20 randomized trials (`fr3_dynamic_randomized.py`, seed 7). Both planners reach the goal in all trials and avoid the obstacle. The distinction is constraint fidelity: the QP maintains hard joint limits in every trial, while the resolved-rate baseline repeatedly exceeds them.

The baseline tends to keep slightly larger clearance because it is unconstrained and can command arbitrarily aggressive joint velocities and accelerations. The QP accepts a smaller but still positive clearance while preserving every declared joint limit.

The timing measurements cover different scopes. The repeated fixed-sparsity study gives 8.34–9.19 ms solve maxima and 12.26–18.48 ms full-cycle maxima; across three randomized-study runs, solve p95 is 6.15–6.66 ms and the maximum is 25.96–33.10 ms. The p95 remains below 10 ms, while the full-cycle and randomized maxima locate the tail that a hard-deadline implementation must reduce.

G. Closed-Loop Dynamics Simulation in MuJoCo

To verify that planner-level differences survive rigid-body contact dynamics, we execute both references on a torque-controlled FR3 in MuJoCo (3.8.1; full mass matrix and gravity), tracking each 100 Hz reference at 500 Hz on the acute-corner maneuver.

MuJoCo computed-torque execution (joint 1 / worst), DI-QP vs. TOTG: peak demanded torque 58.0 vs. 56.7 Nm; torque-rate RMS **181** vs. 383 Nm/s; samples over torque limit 0 vs. 0; tracking RMSE 1.9 vs. 2.0 mrad.

Both stay within the torque envelope and track to $\approx$2 mrad RMS. TOTG's blend-seam acceleration discontinuity manifests as a stepped torque command (torque-rate RMS $2.1\times$ the DI reference) and completes the fixed path faster, while the DI reference is smoother and gentler on the drivetrain—the same smoothness-versus-time trade-off, now in closed-loop dynamics.

VII. Implementation Notes

The QP is solved with OSQP [4]. **Warm-starting** from the previous solution U^* cuts cold-start latency from $\sim$5 ms to under 0.5 ms for the decoupled box-constrained QPs; **sparse structure** is reused throughout, with H assembled once offline and the coupled-obstacle half-space updated in place via a fixed sparsity pattern, avoiding symbolic re-factorization each cycle. $N = 20$ (0.2 s) is the default **horizon length**: $N = 10$ is too short for position-tracking waypoints, while $N \approx 30$ only narrows the static time-optimality gap at higher jerk and solve time. For multi-waypoint tasks the goal is advanced online with no path splicing or re-initialization; online cost is dominated by the QP solve itself ($nN = 140$ variables for the 7-DOF N=20 case), since Φ, Γ are never rebuilt—the practical payoff of the constant-A_d property.

VIII. Discussion

Operating envelope. The method is a **local** predictive planner with four design conditions. **(i)** Position tracking gives time-to-goal of 3–$3.5\times$ TOTG; an analytic velocity-law reference recovers this to 1.06–$1.32\times$ at zero violations. **(ii)** Method-A's Jacobian linearization error grows quadratically in joint-speed$\times$horizon ($\approx$10 cm at 50% speed over 0.2 s), bounding the tolerated horizon and detour size. **(iii)** Method-B (APF) can settle at a local equilibrium, handled by waypoint insertion or switching to Method A. **(iv)** The $N = 20$ (0.2 s) horizon covers the tested obstacle speeds; faster obstacles need a longer horizon or earlier trigger and a terminal braking-to-rest safe set. All evidence is simulation-based (20 randomized FR3 trials; closed-loop MuJoCo rigid-body dynamics, not hardware).

Related work. The path-first baseline (TOTG, TOPP-RA) is time-optimal by construction but rebuilds in batch on a changed path; the present method trades that full-path lookahead for fixed-size reactive updates (its own B2/B4 violations vanish with a C^2 path). Reactive alternatives include DS modulation [12], [13], MPPI [14], and the Path Feasibility Governor [16], which governs a downstream reference for a nonlinear MPC with recursive feasibility – the present planner modifies the reference in the same spirit but drops the path for a fixed-structure QP, directly enforcing hard joint/velocity/acceleration limits rather than the broader non-convex avoidance those methods target. GPU batch optimizers (CuRobo [15]) and trajectory-optimization/whole-body methods [11] instead solve a different, harder problem.

IX. Conclusion

This paper presented a time-domain local Predictive Motion Planner on a virtual double-integrator backbone with constant A_d. Because the prediction matrices are precomputed, online planning reduces to a fixed-structure convex QP that directly enforces velocity, acceleration, and local obstacle constraints while producing C^1 reference trajectories. The experiments show the trade-off clearly: path-first TOTG/TOPP pipelines are faster on static paths, while the DI-QP keeps declared time-domain limits explicit and supports fixed-size reactive updates without a batch rebuild. Future work targets non-convex safe corridors and comparisons with online and sampling-based MPC.

References

[1] I. A. Sucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.

[2] Q.-C. Pham and O. Stasse, "Time-optimal path parameterization for redundantly actuated robots," *IEEE/ASME Transactions on Mechatronics*, vol. 20, no. 6, pp. 3257–3263, 2015.

[3] Y. Cao and J. Tang, "Toward Interaction Dynamics: A Predictive Framework for Safe Physical Human–Robot Interaction," *arXiv:2606.08281*, 2026.

[4] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.

[5] S. Liu *et al.*, "Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-D complex environments," *IEEE Robotics & Automation Letters*, vol. 2, no. 3, pp. 1688–1695, 2017.

[6] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.

[7] H. Pham and Q.-C. Pham, "A new approach to time-optimal path parameterization based on reachability analysis," *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 645–659, 2018.

[8] O. Khatib, "A unified approach for motion and force control of robot manipulators: The operational space formulation," *IEEE Journal on Robotics & Automation*, vol. 3, no. 1, pp. 43–53, 1987.

[9] D. Q. Mayne *et al.*, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[10] T. Kunz and M. Stilman, "Time-optimal trajectory generation for path following with bounded acceleration and velocity," in *Robotics: Science and Systems*, 2012.

[11] J.-P. Sleiman *et al.*, "A unified MPC framework for whole-body dynamic locomotion and manipulation," *IEEE Robotics & Automation Letters*, vol. 6, no. 3, pp. 4688–4695, 2021.

[12] S. M. Khansari-Zadeh and A. Billard, "A dynamical system approach to realtime obstacle avoidance," *Autonomous Robots*, vol. 32, no. 4, pp. 433–454, 2012.

[13] M. Koptev *et al.*, "Reactive collision-free motion generation in joint space via dynamical systems and sampling-based MPC," *The Int. Journal of Robotics Research*, vol. 43, no. 13, pp. 2049–2069, 2024.

[14] G. Williams *et al.*, "Information-theoretic MPC for model-based reinforcement learning," in *IEEE Int. Conf. on ICRA*, 2017, pp. 1714–1721.

[15] B. Sundaralingam *et al.*, "CuRobo: Parallelized collision-free robot motion generation," in *IEEE Int. Conf. on ICRA*, 2023, pp. 8112–8119.

[16] S. Zhang *et al.*, "Integrating planning and predictive control using the path feasibility governor," *arXiv:2507.09134*, 2025.